

NKUA – DIT | K29 – DESIGN AND USE OF DATABASES

Database Design and Queries for Chocolate wholesale company

Dimitrios Kostinis

Introduction

This report documents the implementation phase and assumptions made during the design of the database for the wholesale company “K29-Choco”. It details the implementation of this design into a robust system using the Relational model theory and applying it on the Data Base Management System (DBMS) of PostgreSQL. Also present are some technical challenges encountered and the tactics used to overcome them. Finally, strategies regarding the execution of the 20 assigned SQL queries and comments on some results are presented.

Chapter 1: Database Schema Description

The database of the Choco-K29 wholesale company was designed following the principles of Relational Modeling to ensure data integrity and avoid redundancy. The schema consists of 4 relations:

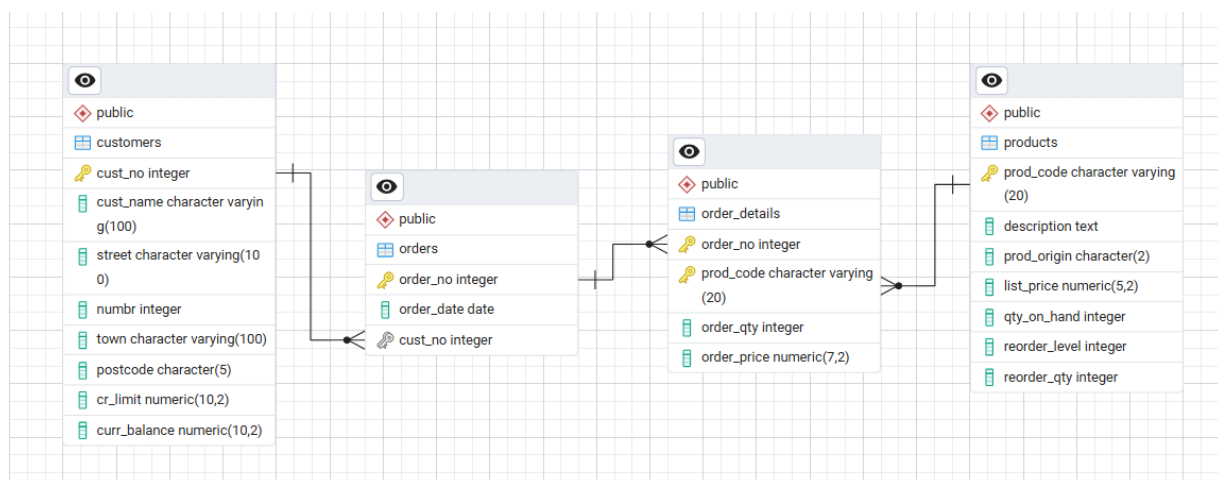
- **Customers:** This relation stores all the necessary B2B customer information regarding their name, business address and location, alongside financial data (credit limit and current debt). The primary key is `cust_no`, their assigned id.
- **Products:** This relation serves as the inventory management table. The primary key is `prod_code`. It stores product relevant details such as place of origin, listed price, current stock and threshold for it, reorder quantity and a short description.
- **Orders:** In this relation ordering events by clients are stored. The primary key is `order_no`. Also included is a foreign key (`cust_no`) referencing the Customers table in a 1:N relationship; 1 customer can place many orders.
- **Order Details:** This relation acts as a relationship table (many-to-many, M:N) between the relations Orders and Products. The (composite) primary key is (`order_no`, `prod_code`), each one referring to Orders and Products accordingly. Also listed are order quantity and price.

Design Assumptions and Constraints

To translate the provided domain specifications into our schema some technical assumptions have been made:

- **Data types:**
 - Financial attributes (cr_limit, curr_balance, list_price, order_price) were modeled using the NUMERIC (p, 2) data type (with p being the assumed transaction upper bound, 2 digits for the fractional part), ensuring decimal precision and avoiding rounding errors.
 - Post code as a 5-character string.
 - Order date, despite requesting the DD-MM-YYYY format, the DATE data type in SQL uses the reverse format. We can align with client's request by formatting the output in query stage.
- **Domain Constraints:**
 - Order Quantity: As dictated, the order quantity must be greater or equal to 10 kg. This was enforced with a CHECK constraint.
 - Product origin: Another CHECK constraint to only accept predefined geographical codes 'SA', 'CA', 'WA', 'EA' and 'AS'.
- **Pricing:** I decided to separate the concepts of product list_price and order_price to account for the client-specific discounts; the second ceases to be a linear equation of the first.
- **String length:** While the initial string types had the standard upper limit of 45 characters, when loading the CSV files, I noticed this formatting had to be increased to prevent data truncation.

Figure 1: The Entity-Relationship Diagram



Chapter 2: Challenges in table-populating stage:

During the database population phase, several data integrity and formatting inconsistencies were noticed between the provided CSV files and the logical schema. The following technical solutions were implemented to maintain the data integrity:

- **String Length Truncation:** While the initial string types were given a standard upper limit (e.g. VARCHAR(45)), during the CSV loading phase, I noticed this length had to be increased for specific fields to prevent data truncation errors from long entries in the raw data.
- **Loading order:** During the initial database population phase, foreign key violations occurred when importing the Order details table. This led me to the conclusion that orphan records (without a parent in Orders) have triggered referential integrity issues. However, later investigations showed no such record existed. A more probable explanation was an incomplete population in the Orders table that I resolved later. Understanding this hierarchy was crucial for the continuation of this project and for the ones to come after it.
- **Staging and Casting Process:** The direct import of the CSV files via the pgAdmin failed on many occasions due to delimiter mismatches, encoding problems, trailing spaces in `CHAR 5` domains (like postcode), unnecessary single quotes on strings and formatting issues in prices (commas instead of decimal points). To fix this issue, the raw data was first loaded into a temporary Staging table with text domains and then cast to the appropriate data type using the REPLACE() function and then loaded on to the proper tables.

Overcoming these challenges provided valuable practical experience in data cleaning, adaptation and a great introduction in staging environments and ETL (Extract, Transform, Load) processes. As a result, 100 entries on Products, 1000 on Customers, 4900 Orders and 14752 Order Details were successfully imported on to our tables.

Chapter 3: SQL Query Methodology and Observations

This chapter outlines some logical assumptions and techniques applied on the 20 requested SQL queries to retrieve mathematically accurate and consistent relevant data.

1. **Monetary Value:** For all financial queries, monetary value was defined as the product quantity times the agreed upon price (`order_qty * order_price`). This accounts for customer specific discounts, as required within our parameters.
2. **Queries regarding Date:** The initial string-matching operations revealed a weakness (also appearing in `order_date` definition). A client on an American and another on European computer would not get the same answer due to

different date formatting, thus making this query inconsistent. To resolve this, we treated them as mathematical ranges (using comparison operators), thus creating custom time intervals.

Some structural observations revealed include:

- The 1:N relationship between Orders and Order Details, as 1 entry on orders maps to, on average, 3 on order details (1 placed order on average includes 3 products).
- Queries concerning “Almond-Choco” and “Orange-Choco” products initially yielded no results due to single quote encapsulation so insensitive pattern matching was implemented (ILIKE and wildchars).
- Query 15 proved the need for separation of listed price and order price.
- Null Handling: An integrity audit revealed order records containing `NUL` customer references (order 108), which caused `NOT IN` containing queries to return zero rows due to SQL's 3-Valued-Logic, as any comparison with nulls returns `UNKNOWN`. To solve this we utilized Left Anti-Joins (`LEFT JOIN ... WHERE key IS NULL`) on anti-filtering queries (12, 16, 17).